

Design and Analysis of Cost-Cognizant Test Case Prioritization Using Genetic Algorithm with Test History

Yu-Chi Huang¹, Chin-Yu Huang^{1,2}, Jun-Ru Chang² and Tsan-Yuan Chen³

¹*Department of Computer Science*

³*Media Creation Department*

²*Institute of Information Systems and Applications*

CyberLink Corp.

National Tsing Hua University

Taipei, Taiwan

Hsinchu, Taiwan

Abstract — During software development, regression testing is usually used to assure the quality of modified software. The techniques of test case prioritization schedule the test cases for regression testing in an order that attempts to increase the effectiveness in accordance with some performance goal. The most general goal is the rate of fault detection. It assumes all test case costs and fault severities are uniform. However, those factors usually vary. In order to produce a more satisfactory order, the cost-cognizant metric that incorporates varying test case costs and fault severities is proposed. In this paper, we propose a cost-cognizant test case prioritization technique based on the use of historical records and a genetic algorithm. We run a controlled experiment to evaluate the proposed technique's effectiveness. Experimental results indicate that our proposed technique frequently yields a higher Average Percentage of Faults Detected per Cost (APFDc). The results also show that our proposed technique is also useful in terms of APFDc when all test case costs and fault severities are uniform.

Keywords-regression testing; cost-cognizant test case prioritization; test cost; fault severity; rate of fault detection; Average Percentage of Faults Detected per Cost(APFDc)

I. INTRODUCTION

Regression testing is used to validate the modified software product to ensure its quality. The simplest approach is to reuse all test cases in the existing test suites. Nevertheless, this can be very expensive and impracticable because it may need numerous test case executions [1]. Other approaches have therefore been proposed to reduce the cost of regression testing [2].

Test case prioritization approaches schedule test cases in an order so that those with a higher priority will be executed earlier [3]. Testers often order test cases to meet some performance goal. The most general goal is rate of fault detection. Many techniques have been proposed to prioritize test cases for regression testing [3][4][5]. These techniques analyze source code to get useful information, such as function coverage of test cases, and then prioritize test cases according to this information. However, these techniques have two limitations. First, these techniques consider that test case costs and fault severities are uniform, while they usually vary and these factors may impact the effectiveness of regression testing. Second, if testers cannot obtain the source code, applying these techniques will be difficult because coverage information is hard to acquire. The cost-cognizant test prioritization approaches and history-based test prioritization approaches have therefore been proposed to overcome

this limitation [6][8].

A genetic algorithm (GA) that simulates the process of evolution is a search algorithm used to solve optimization and search problems. In this paper, we propose a cost-cognizant test case prioritization technique, which is based on the use of historical test case execution records and a GA. We gathered the test costs, fault severities, and detected faults of each test case from the latest regression testing and then used a GA to find an order with the greatest rate of “units of fault severity detected per unit test cost.”

The cost-cognizant metric, Average Percentage of Faults Detected per Cost (APFDc), has been proposed to evaluate the effectiveness of the cost-cognizant test case prioritization techniques [10]. We used this metric to evaluate the effectiveness of the proposed technique and then compared the proposed technique with five existing techniques, and two techniques served as an experiment control. Further, uniform test case and fault severity were also included in the experiment to investigate their impact on the effectiveness of the techniques. The experiment results show that our proposed technique can improve the effectiveness of cost-cognizant test case prioritization without analyzing the source code, no matter whether test case costs and fault severities are uniform or not.

II. BACKGROUND AND RELATED WORK

A. Cost-Cognizant Test Case Prioritization

Test case prioritization approaches sort existing test cases for regression testing in accordance with some performance goals [3]. In other words, the test case prioritization problem is a kind of search problem, and test case prioritization techniques search test orders to find an order that meets the goal. The most general goal is the rate of fault detection, i.e., testers wish to detect faults as soon as possible. Numerous techniques have been proposed to meet this goal [3][4][7]. Most of them rely on the assumption that all test case costs and fault severities are equal, but these factors usually vary under different conditions; therefore, those prioritization techniques may produce inappropriate test orders.

In order to generate a more satisfactory testing order for regression testing, cost-cognizant test case prioritization approaches and a cost-cognizant metric have been proposed. Cost-cognizant test case prioritization approaches schedule test cases according not only to the coverage information but also to the varying test case costs and fault severities of each test

case [6][8].

B. Related Works

1) *Time-Aware Test Suite Prioritization*: This technique utilizes a GA to search for the optimum test tuple that has the highest potentiality of fault detection for the time-aware prioritization [5]. The fitness function, fit , which is used to quantify the potentiality of fault detection of a test tuple, has been defined as follows:

$$fit(P, \sigma_j, tc, w) = F_{pri}(P, \sigma_j, tc, w) + F_{sec}(P, \sigma_j, tc) \quad (1)$$

$$F_{pri}(P, \sigma_j, tc, w) = cc(P, \sigma_j, tc) \times w \quad (2)$$

$$F_{sec}(P, \sigma_j, tc) = \frac{F_{s-actual}(P, \sigma_j, tc)}{F_{s-max}(P, \sigma_j, tc)} \quad (3)$$

$$F_{s-actual}(P, \sigma_j, tc) = \sum_{i=1}^{|\sigma_j|} time(\langle T_i \rangle) \times cc(P, \sigma_{j\{1,i\}}, tc) \quad (4)$$

$$F_{s-max}(P, \sigma_j, tc) = cc(P, \sigma_j, tc) \times \sum_{i=1}^{|\sigma_j|} time(\langle T_i \rangle) \quad (5)$$

In equations (1) to (5), P is the tested program; σ_j is the test tuple j whose fitness value will be evaluated; tc is the test adequacy criteria, such as function coverage or block coverage, used to determine the fitness value of σ_j ; and w is a program coverage weight. The function, $cc(P, \sigma_j, tc)$, returns the coverage percentage of the entire test tuple σ_j . The value of w should be large enough so that F_{pri} dominates the result of the fit function. $time(\langle T_i \rangle)$ represents the execution time of i -th test case of test tuple σ_j . The $\sigma_{j\{1,i\}}$ represents the subtuple of σ_j .

This technique has some limitations. First, its effectiveness for cost-cognizant test prioritization was not verified. Second, the time overhead for calculating the fitness value of all test tuples is high because it may need lots of coverage evaluations. Third, this technique prioritizes test cases according to the coverage information of each test case. The prioritization may become difficult if testers cannot obtain source code.

2) *Historical Value-Based Approach*: In [8], H. Park, H. Ryu, and J. Baik proposed a historical value model, which is used to quantify the historical value of a test case based on the historical test case costs and the historical fault severities for cost-cognizant test prioritization.

The historical value model has been defined as follows:

$$HV_{(t,i)} = [100 + \min(C_{(t,1)}, C_{(t,n)}) - C_{(t,i-1)}] \times wC_{i-1}, \quad (6)$$

$$+ FS_{(t,i-1)} \times wFS_{i-1} \quad (7)$$

$$wC_i = \frac{\bar{C}_i}{C_i + FS_i},$$

and

$$wFS_i = \frac{\bar{FS}_i}{C_i + FS_i}. \quad (8)$$

In equations (6) to (8), $HV_{(t,i)}$ includes the historical value of test case t used at i -th regression testing. $C_{(t,i-1)}$ is the relative cost of test case t at i -1th regression testing, which is calculated by comparing it against the maximum test case cost in the test suite. $FS_{(t,i-1)}$ is the relative total fault severities of test case t at i -1th regression testing. The function, $\min(C_{(t,1)}, C_{(t,n)})$, returns the minimum relative cost value of test case t between its relative cost

at the first regression testing and its relative cost at the n -th regression testing. wC_{i-1} and wFS_{i-1} are two weight factors at the i -1th regression testing. $\bar{C}_i$ and $\bar{FS}_i$ represent the mean value of the relative cost of all test cases and the mean value of the relative total fault severity of all test cases, respectively.

This technique, however, does not take the test case similarity into account. In other words, test cases that have the same historical value as calculated by the model may have the same capability to detect faults; therefore, giving them the same rank may affect the effectiveness of the regression testing.

C. Metrics of Prioritization Effectiveness

In order to evaluate the effectiveness of the prioritization techniques, the Average Percentage of Fault Detected (APFD) metric has been proposed [3][4]. APFD is used to measure the rate of fault detection. The definition of APFD is based on the assumption that all test case costs are uniform and all fault severities are uniform. However, these factors are usually diverse, and APFD may generate inappropriate results in such situations. In order to overcome this limitation, the cost-cognizant metric, the APFDc, has been proposed [6][9]. This metric incorporates varying test case costs and fault severities into APFD. In other words, if all fault severities and test case costs are uniform, the formula for APFDc will reduce to the formula for APFD.

III. PROPOSED TECHNIQUE

We propose a cost-cognizant test case prioritization to overcome the limitations of the existing test case prioritization techniques described in Section II. The problem that the proposed technique attempts to solve is described as follows:

Definition: The History-Based Test Case Prioritization

Given: T , a test suite; $P(T!)$, the set of all permutations of T ; HR , the historical records of all elements in $P(T!)$; f , a function from $P(T!)$ to real numbers; fit , a function from HR to real numbers.

Problem:

$$\text{Find } T_{max} \in P(T!) \text{ such that } \forall T' \in P(T!), (T' \neq T_{max}), \\ [fit(HR_{T_{max}}) \geq fit(HR_{T'})] \text{ and } [f(T_{max}) \geq f(T')] \quad (9)$$

$P(T!)$ represents all possible orders of the test cases in T , HR is the historical records of the orders in $P(T!)$. The function, fit , evaluates the award value of the order based on its historical record, and the f function quantifies the effectiveness of the order. The goal of the problem is to find the order that demonstrates goodness in the past and also effectiveness in the future regression testing.

The objective of the proposed technique is to search for an order of the existing test suite which has the greatest effectiveness in terms of cost-cognizant test case prioritization in the preceding regression testing. Since this search problem, which is a kind of NP-hard problem, can be solved by the GA [12], we input the historical execution information of test cases to a GA to produce

such an order.

When prioritization is applied, the proposed technique acquires the stored historical information of each test case and inputs this data to the GA to produce an order that has the greatest effectiveness in terms of the preceding regression testing. After applying regression testing, the execution results of each test case, which will become historical information, are stored in the Historical Information Repository.

Algorithm TestCasePrioritization
Input:
 T : Test suite
 p : Population size
 g : Number of generation
 cp : Crossover probability
 mp : Mutation probability
Output:
 T_{greatest} : A test order which has greatest fitness value in population in final generation
1. Acquire test cost list, tc , detected fault list, fl , and fault severity list, fsl , from Historical Information Repository
2. $P_1 \leftarrow \text{genPopulation}(T, p, fl, fsl)$
3. **for** $i = 1$ **to** g
4. $F_i \leftarrow \text{evaFitness}(P_i, tc, fl, fsl)$
5. $P_{i+1} \leftarrow \text{addTwoBest}(F_i, P_i)$
6. **for** $j = 3$ **to** p
7. $\text{parent}_1 \leftarrow \text{randSelectParent}(P_i)$
8. $\text{parent}_2 \leftarrow \text{randSelectParent}(P_i)$
9. $\text{child}_1, \text{child}_2 \leftarrow \text{crossover}(\text{parent}_1, \text{parent}_2, cp)$
10. $\text{child}_1 \leftarrow \text{mutation}(\text{child}_1, mp)$
11. $\text{child}_2 \leftarrow \text{mutation}(\text{child}_2, mp)$
12. $P_{i+1} \leftarrow \text{addChildren}(\text{child}_1, \text{child}_2)$
13. $F_{g+1} \leftarrow \text{evaFitness}(P_{g+1}, tc, fl, fsl)$
14. $T_{\text{greatest}} \leftarrow \text{selectBestChild}(F_{g+1}, P_{g+1})$
15. **return** T_{greatest}

Figure 1. The GA to prioritize test case

It is noticed that the GA described in Fig. 1 schedules test cases in test suite T on the basis of their historical information, such as test cost, detected faults, and fault severities. On line 1, the algorithm acquires the test cost list, tc , detected fault list, fl , and fault severity list, fsl , from the Historical Information Repository. The test cost list and detected fault list respectively record the cost of each test case and the detected faults of each test case in the preceding regression testing. The fault severity list records the severity of each fault in the preceding regression testing. On line 2, the first population which contains p chromosomes is generated by genPopulation . The encoding of chromosomes is permutation encoding. Each chromosome represents the execution order of test cases.

First, genPopulation calculates the sum of detected fault severities for each test case and sorts test cases in decreasing order according to that sum. This decreasing order is added to the first population because we want the test order to find critical faults that have higher severity as soon as possible, and ensure that the effectiveness of the final order produced by $\text{TestCasePrioritization}$ is at a minimum better than this order. Finally, genPopulation generates $p-1$ test case orders and adds them to the first population.

In the **for** loop beginning on line 3, the evolution

process is performed. The algorithm evaluates the fitness value of each chromosome on line 4. On line 5, the algorithm adds two chromosomes that have the best fitness value to the new population. These chromosomes are added to the new population to ensure that at least two best chromosomes survive, and the best chromosome in the new population is at least better than these chromosomes.

In the inner loop beginning on line 6, the reproduction process is performed. The algorithm selects two parents on line 7 and line 8 by the randSelectParent . The parent is selected by the roulette wheel selection method. After selecting two parents, the crossover may recombine the selected parents based on the crossover probability, cp , on line 9. On lines 10 and 11, the mutation mutates the child on the basis of the mutation probability. On line 12, the algorithm adds two new chromosomes generated by the reproduction process to the new population. The reproduction process on lines 6-12 is repeated until the new population has p chromosomes. The evolution process on lines 3-12 is applied g times to evolve the population. When the final population, P_{g+1} , is produced, the chromosome whose fitness value is the greatest is selected to be the test order.

A. Crossover

Crossover is a recombination process that combines the segments of one chromosome with the segments of another. The new chromosomes produced by crossover inherit some characteristics from both parents. The crossover algorithm is the single point crossover. It may be applied according to the crossover probability, cp . If crossover is applied, crossover selects crossover points, p_1 and p_2 , for parent_1 and parent_2 , respectively. The subsequences before the crossover point are then copied from both parents. The combined function generates a child by combining the copied subsequence of one parent with genes of another parent that are not in the copied subsequence. Otherwise, the child is a duplicate of the parent.

B. Mutation

The mutation operator is used to retain the diversity of chromosomes in the new population. It may be applied according to the mutation probability, mp . If the algorithm is applied, mutation selects two child genes randomly and switches their positions. Otherwise, the mutation operator is not applied. For test case prioritization on the basis of historical information, some test cases that had less fault detection effectiveness in previous regression testing may have better fault detection effectiveness in future regression testing. The mutation algorithm also gives those test cases a chance to gain higher priority.

C. Fitness Function

In Fig. 1, the evaFitness on lines 4 and 13 evaluates a fitness value of each chromosome in the population through the fitness function. Since the goal of our proposed technique is to search out an order of the existing test suite that has the greatest effectiveness in

terms of cost-cognizant test case prioritization in the preceding regression testing, *evaFitness* utilizes historical information to calculate the fitness value according to the APFDc.

If there are m faults and n test cases for a tested program in the preceding regression testing, the fitness function, f , is defined as follows:

$$f = \frac{\sum_{i=1}^m (hf_i \times (\sum_{j=TF_i}^n ht_j - \frac{1}{2} ht_{TF_i}))}{\sum_{j=1}^n ht_j \times \sum_{i=1}^m hf_i} \quad (10)$$

In equation (10), hf_i is the fault severity of fault i in the preceding regression testing, and ht_j is the cost of j -th test case in the chromosome. TF_i is the TF_i -th test case in the chromosome that detects the fault i first.

IV. EXPERIMENTS AND DISCUSSION

A. Experiment Instrumentation

In our experiment, we focused on the version-specific test case prioritization [3], which prioritizes test cases for a specific modified version of a program. We obtained five sequential versions and one test suite for the tested program, *sed*, from [11].

TABLE I. TESTED PROGRAM – *sed*

Version	Lines of Code	Number of Functions	Number of Faults	Number of Test Cases
1	11990	178	5	360
2	10055	120	6	360
3	6671	259	4	360
4	6704	278	6	360
5	7164	285	4	360

TABLE II. TEST CASE PRIORITIZATION TECHNIQUES

Abbreviation	Descriptions
random	order test case randomly
optimal	the optimal solution for test case prioritization problem
hist-fault-detected [7]	prioritize historical fault detected effectiveness
GA-hist	proposed technique
hist-value [8]	prioritize historical value considering cost and severity
GA-fun [5]	prioritize by genetic algorithm using coverage of functions
total-fun [3]	prioritize coverage of functions
cc-total-fun [6]	prioritize coverage of functions considering cost and severity

1) *Programs and Comparator Techniques*: The information of the tested program used in the experiment is shown in Table I. In order to evaluate our proposed technique, we considered other different test case prioritization techniques which are listed in Table II. The random and optimal techniques served as experimental controls. The GA-hist, hist-value and hist-fault-detected (historical fault detection effectiveness prioritization) are history-based techniques which prioritize test cases according to their historical information. The cc-total-fun (cost-cognizant total function coverage prioritization), total-fun (total function coverage prioritization), and GA-fun are coverage-based techniques which schedule test cases on the basis of their function coverage. Further,

the GA-hist, hist-value, and cc-total-fun techniques are also developed for cost-cognizant test case prioritization. GA-hist is our proposed technique. GA-fun is the time-aware test case prioritization technique, and Hist-value is the cost-cognizant test case prioritization technique. These two techniques have been described in Section II(B) in detail.

2) *Test Case Cost and Fault Severity Design*: Because neither tested program provided the cost of each test case nor the severity of each fault, we created such information. In our experiment, we measured the test case cost in terms of test case execution time. This is a criterion commonly used in previous research [6][8][9]. Further, we assigned the fault severity according to the function criticality presented in [6][9].

Since the historical information was not available when prioritizing test cases for the first version of the tested programs, we made it for the history based techniques. In this situation, the cost of each test case is set as 1 and the total detected fault severity of each test case is set as 0.

TABLE III. PARAMETERS USED BY GA-fun

Parameter	Value
p (population size)	200
g (generation)	400
cp (crossover probability)	1
mp (mutation probability)	0.7
tc (test adequacy criteria)	Function
w (program coverage weight)	100

TABLE IV. PARAMETERS USED BY GA-hist

Parameter	Value
p (population size)	200
g (generation)	400
cp (crossover probability)	1
mp (mutation probability)	0.7

B. Effectiveness Comparisons and Discussion

Because random, GA-hist, and GA-fun could generate different test case orderings for regression testing, 1000 prioritizations were created by each of them for each regression testing. The average results of these prioritizations were then calculated.

We compare the effectiveness of the techniques in Table II in two conditions. In condition 1, the cost and total detected fault severity of the test cases are various. In condition 2, the cost and total detected fault severity of the test cases are uniform. Furthermore, the parameters used by GA-fun and GA-hist are shown in Table III and Table IV.

1) *Condition 1*: In this condition, we consider a variety of test case costs and fault severities. Since historical information was not available when ordering test cases for the first regression testing of the tested programs, we created it as described in Section IV(A), part 2. In this situation, GA-hist will degrade to a near random prioritization technique, and other prioritization techniques that also utilize historical information, including hist-fault-detect, hist-value and cc-total-fun, will degrade to no prioritization because all test cases'

priority calculated by these techniques will be uniform. Table V shows the APFDc value of each technique from the experimental results of each version of sed.

First, we consider the mean APFDc values of the prioritization techniques in the experiment. As seen from Table V, GA-hist has a greater mean APFDc value than the other techniques. Comparing GA-hist with random, GA-hist is a significant improvement on mean APFDc. GA-hist also performs well in comparison to history-based prioritization techniques such as hist-fault-detect and hist-value. In contrast with coverage-based prioritization techniques such as GA-fun, total-fun, and cc-total-fun, on average, GA-hist improves 2% to 4%, but is a significant improvement in comparison to total-fun on sed. Moreover, cc-total-fun, which is a cost-cognizant technique, has a higher performance than total-fun, which does not consider test case costs and fault severities when prioritizing.

Next, we consider the experimental results of each version. It is a surprise that GA-hist provides high performance for the first regression testing. GA-hist improves the APFDc value on average. On version 2, although GA-hist shows a lower APFDc value in comparison to cc-total, it is insignificant. We also find that some techniques such as hist-value and hist-fault-detect have greater APFDc values than GA-hist on version 4. It results from the fault exposed by a few test cases whose historical records are worse or the same as other test cases on this version. For instance, the fault #6 is detected by test cases #34, #321, and #339 on version 4 in the experiment. According to the historical

information, we find that test cases #321 and #339 did not detect any faults on version 3.

In addition, there are 17 test cases that have the same fault detection capability as the test case #34. Because hist-value and hist-fault-detect give the same rank to test cases that have the same capability, test case #34 and those 17 test cases will be executed earlier, while test cases #321 and #339 will be executed later. However, since GA-hist randomly orders the test cases that have the same ability, test cases #34, #321 and #339 may get higher or lower priority individually. If they all get lower priority, the fault 6 will be exposed later, and such an order will obtain a lower APFDc value. The average APFDc value of 1000 prioritizations created by GA-hist is therefore lower than the APFDc value of hist-value and hist-fault-detect.

2) *Condition 2*: In this condition, the APFDc metric is equivalent to the APFD metric. The experimental results are shown in Table VI. Again, we consider the mean APFDc value of the prioritization techniques first. GA-hist also provides higher effectiveness than the other techniques, on both sed and flex. Next, we consider the experimental results of each version. GA-hist also obtains a lower APFDc value than cc-total and hist-fault-detect on versions 2, and gets the highest APFDc value on the other versions. Further, the effectiveness of history-based techniques such as hist-value and hist-fault-detect is lower than the coverage-based techniques on version 3. We also find that even though the historical information may be useless on this version, GA-hist can provide better performance than the other techniques.

TABLE V. APFDc VALUES OF THE PRIORITIZATION TECHNIQUES FOR TESTED PROGRAM IN CONDITION 1

Version	1	2	3	4	5	Mean	Improvement
random	73.26%	73.09%	76.89%	79.69%	71.69%	74.92%	+14.14 %
optimal	99.63%	99.59%	99.78%	99.68%	99.66%	99.67%	-10.61 %
hist-fault-detect	78.92%	75.94%	46.51%	97.03%	71.44%	73.97%	+15.09 %
GA-hist	85.26%	91.17%	97.11%	90.28%	81.49%	89.06%	--
hist-value	78.92%	78.24%	82.29%	94.12%	80.92%	82.90%	+6.16 %
GA-fun	84.87%	87.66%	90.17%	89.27%	80.57%	86.51%	+2.55 %
total-fun	82.62%	79.75%	67.93%	82.63%	65.45%	75.68%	+13.38 %
cc-total-fun	77.64%	92.16%	92.59%	90.35%	76.41%	85.83%	+3.23 %

TABLE VI. APFDc VALUES OF THE PRIORITIZATION TECHNIQUES FOR TESTED PROGRAM IN CONDITION 2

Version	1	2	3	4	5	Mean	Improvement
random	76.99%	80.38%	64.55%	74.88%	79.91%	75.34%	+16.00 %
optimal	99.67%	99.47%	99.63%	99.81%	99.61%	99.64%	-8.30 %
hist-fault-detect	85.03%	82.50%	55.97%	97.36%	78.13%	79.80%	+11.54 %
GA-hist	89.39%	91.16%	96.49%	94.23%	85.43%	91.34%	--
hist-value	85.02%	82.50%	71.80%	94.40%	73.61%	81.47%	+9.87 %
GA-fun	88.60%	90.66%	91.46%	93.63%	82.86%	89.44%	+1.09 %
total-fun	88.81%	86.90%	74.51%	92.22%	72.50%	82.99%	+8.35 %
cc-total-fun	82.69%	92.87%	86.53%	96.75%	78.40%	87.45%	+3.89 %

V. IMPACT OF NUMBER OF GENERATIONS AND POPULATION SIZE

A. Efficiency Comparison and Analysis

In this section, we compare the efficiency of GA-hist with the efficiency of GA-fun; both of them are GA-produced prioritization techniques. We measure the efficiency in terms of the execution time. We compare their efficiency when various parameters, such as the number of generations and

the population size, are applied. The experimental results are shown in Table VII. The second column indicates the various combinations of number of generation and the population size. The third and fourth columns show the average execution time, calculated by averaging the execution time of each version.

As seen from Table VII, we find that (1) the smaller the population size, the lower the average execution time of both GA-produced prioritizations, and (2) the results indicate that

GA-hist is much more efficient than GA-fun. This results from the time overhead consumed by calculating the fitness of each chromosome. Further, generating the first population also affects the efficiency. As a result, although both the (100, 400) configuration and the (400, 100) configuration should calculate fitness 40,000 times, the (400, 100) configuration has lower execution time.

Furthermore, assume the existing test suite contains m test cases, and a tested program contains k functions and n faults. Then, evaluating the function coverage of an order test case requires a cost $O(mk)$, and searching the first test case in an order that detects the specific fault has a cost $O(m)$. According to equations (1) to (5), the cost of the fitness calculator of GA-fun is $O(m^2k)$. Moreover, since GA-hist calculates the fitness of a chromosome on the basis of equation (10), it needs a cost $O(m^2n)$. When the following relation holds, GA-hist is more efficient than GA-fun.

$$k = a \times n^c, \text{ where } a \geq 1 \text{ and } c > 1 \quad (11)$$

While this relation holds in this experiment, the results show that GA-hist is much more efficient than GA-fun.

TABLE VII. THE AVERAGE EXECUTION TIME OF GA-hist AND GA-fun

Program	(# of generations, population size)	Average Execution Time (second)	
		GA-hist	GA-fun
sed	(100, 400)	27.26	204.02
	(250, 250)	42.86	317.82
	(400, 100)	25.56	154.22

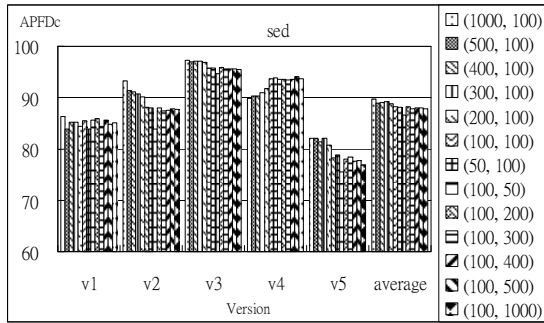


Figure 2. The trend of APFDc values

B. Effectiveness Analysis

In this section, we discuss the influence of the number of generations and population size on APFDc values. The change trends among the APFDc values are revealed in Fig. 2. First, let's consider the APFDc value of each version. When the number of generations is greater than the population size, the proposed technique provides a higher APFDc value. The relationship is reversed, however, for version 4. The number of generations implies the number of times the crossover operation is executed. The more occasions on which crossover is performed, the greater the chance that test cases with poorer historical records in terms of their potential ability to detect faults in the future will gain a higher priority. Accordingly, the proposed technique leads to a higher APFDc if the number of generations is greater.

On version 4, we find the faults exposed by a few test cases. In this situation, applying the crossover more times may have a negative influence because these test cases have

a greater chance to get a lower priority. The proposed technique therefore provides a lower APFDc for this version when the number of generations is greater.

Finally, we consider the average APFDc values when various numbers of generations and population sizes are applied. The results also show that the greater the number of generations, the more effective the proposed technique.

VI. CONCLUSIONS

In this paper, we propose a cost-cognizant prioritization technique that orders test cases according to their historical information. The proposed technique prioritizes test cases according to their various test costs and fault severities without analyzing the source code. The proposed technique also avoids situations where the test cases with equivalent ability in the previous regression testing are given the same rank, improving the prioritization performance.

In the experiment, we used a UNIX utility program, sed, to evaluate the proposed technique. According to the results, we confirm that the proposed technique is useful. We also confirm that prioritizing test cases on the basis of their historical information can provide high effectiveness. In the future, we will continue to improve the effectiveness of regression testing by incorporating other historical information, or even by combining history-based techniques with coverage-based techniques.

VII. ACKNOWLEDGMENTS

The work described in this paper was supported by the National Science Council, Taiwan, under Grant NSC 97-2221-E-007-052-MY3 and NSC 98-2221-E-007-067.

REFERENCES

- [1] J. M. Kim, A. Porter, and G. Rothermel, "An Empirical Study of Regression Test Application Frequency," *Proc. 22nd Int'l Conf. Software Eng.*, pp. 126-135, June 2000.
- [2] D. Binkley, "Semantics Guided Regression Test Cost Reduction," *IEEE Trans. Software Eng.*, pp. 498-516, Aug. 1997.
- [3] G. Rothermel, R. H. Untch, Chu Chengyun, and M.J. Harrold, "Prioritizing Test Cases for Regression Testing," *IEEE Trans. Software Eng.*, pp. 929-948, Oct. 2001.
- [4] S. Elbaum, A. G. Malishevsky, and G. Rothermel, "Test Case Prioritization: A Family of Empirical Studies," *IEEE Trans. Software Eng.*, pp. 159-182, Feb. 2002.
- [5] K. R. Walcott, M. L. Soffa, G. M. Kapfhammer, and R. S. Roos, "Time-Aware Test Suite Prioritization," *Proc. Int'l Conf. International Symposium on Software Testing and Analysis*, pp. 1-12, July 2006.
- [6] A. G. Malishevsky, J. R. Ruthruff, G. Rothermel, and S. Elbaum, "Cost-cognizant Test Case Prioritization," *Technical Report TR-UNL-CSE-2006-0004*, University of Nebraska- Lincoln, March 2006.
- [7] J. M. Kim, and A. Porter, "A History-Based Test Prioritization Technique for Regression Testing in Resource Constrained Environments," *Proc. 24th Int'l Conf. Software Eng.*, pp. 119-129, May 2002.
- [8] H. Park, H. Ryu, and J. Baik, "Historical Value-Based Approach for Cost-cognizant Test Case Prioritization to Improve the Effectiveness of Regression Testing," *Proc. 2nd Int'l Conf. Secure System Integration and Reliability Improvement*, pp. 39-46, July 2008.
- [9] S. Elbaum, A. Malishevsky, and G. Rothermel, "Incorporating Varying Test Costs and Fault Severities into Test Case Prioritization," *Proc. 23rd Int'l Conf. Software Eng.*, May 2001.
- [10] Z. Li, M. Harman, and R. M. Hierons, "Search Algorithms for Regression Test Case Prioritization," *IEEE Trans. Software Eng.*, pp. 225-237, Apr. 2007.
- [11] Software-artifact Infrastructure Repository, <http://sir.unl.edu>.
- [12] Z. Li, M. Harman, and R. M. Hierons, "Search Algorithms for Regression Test Case Prioritization," *IEEE Trans. Software Eng.*, pp. 225-237, Apr. 2007.